

Serverless Webbapplikation - Rapport

Projekt: Skalbar värdmiljö - Serverless webapplikation

Student: Maria Schillström

Datum: November 2025

Innehållsförteckning

- [Serverless Webbapplikation - Rapport](#)
- [Innehållsförteckning](#)
 - [1. Arkitektur](#)
 - [1.1 Översikt](#)
 - [1.2 Arkitekturdiagram](#)
 - [1.3 Dataflöde](#)
 - [1.4 Skalbarhet](#)
 - [2. Säkerhet](#)
 - [2.1 S3 Bucket Security](#)
 - [2.2 API Gateway Security](#)
 - [2.3 Lambda Security](#)
 - [2.4 CORS-konfiguration](#)
 - [3. Infrastructure as Code](#)
 - [3.1 Metodik](#)
 - [3.2 CloudFormation Templates](#)
 - [3.3 Deployment](#)
 - [4. Kostnadsanalys](#)
 - [4.1 Estimerad månadskostnad \(låg trafik\)](#)
 - [4.2 Skalningskostnader](#)
 - [5. Förbättringsområden](#)
 - [5.1 Säkerhet](#)
 - [5.2 Prestanda](#)
 - [5.3 Monitoring](#)
 - [5.4 CI/CD](#)
 - [6. Slutsats](#)
 - [7. Referenser och Dokumentation](#)

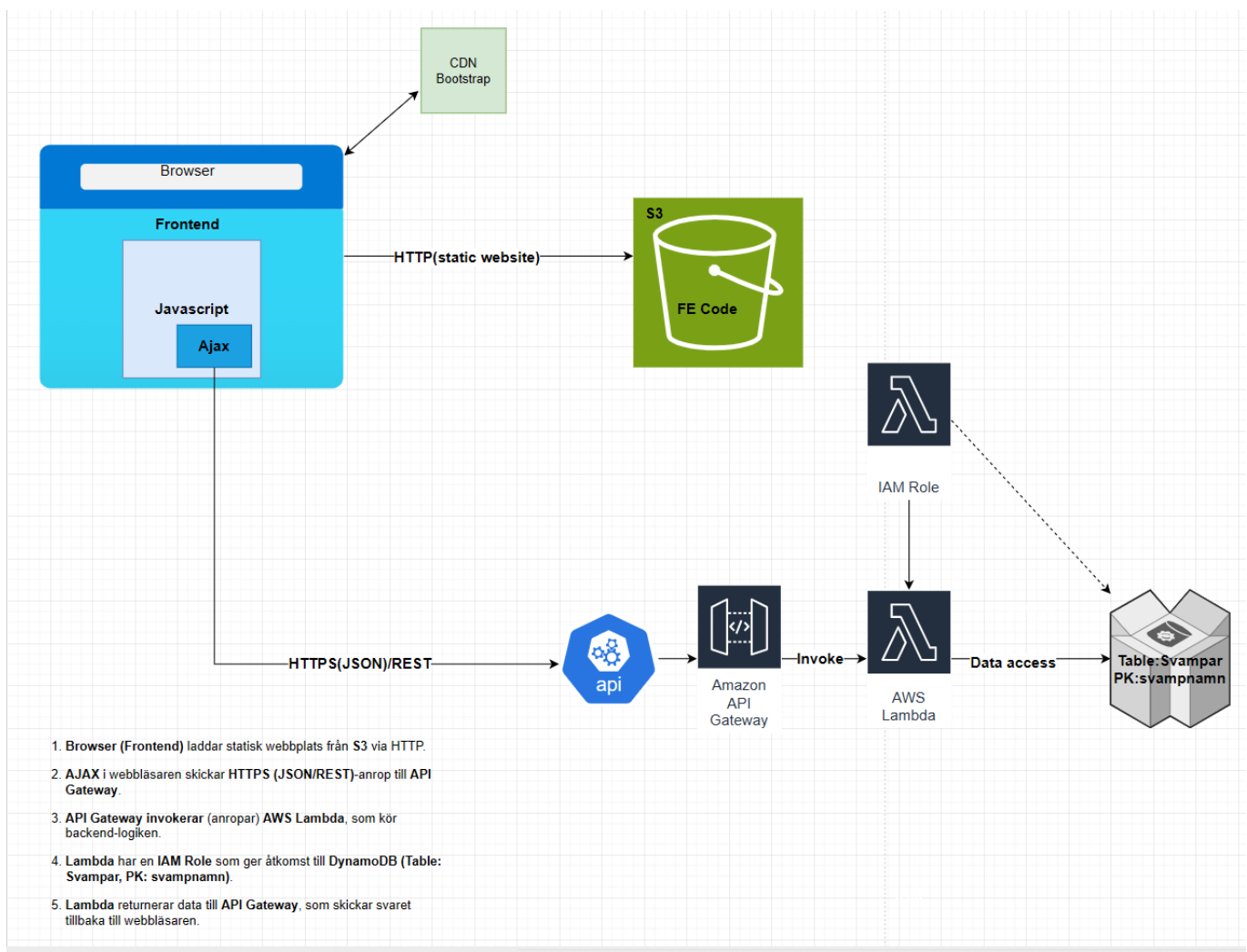
1. Arkitektur

1.1 Översikt

Applikationen följer en serverless arkitektur med följande komponenter:

- **S3 Bucket:** Hostar den statiska webbsidan (HTML, CSS, JavaScript)
- **API Gateway:** REST API som exponerar endpoints för frontend
- **Lambda Function:** Serverless backend-logik som exekveras on-demand
- **DynamoDB:** NoSQL-databas för att lagra applikationsdata
- **IAM Roles:** Hanterar behörigheter mellan tjänsterna

1.2 Arkitekturdiagram



1.3 Dataflöde

1. Användaren navigerar till S3 website endpoint
2. Webbbläsaren laddar `index.html` från S3
3. JavaScript i sidan gör ett HTTPS-anrop till API Gateway
4. API Gateway triggar Lambda-funktionen
5. Lambda läser/skriver data från/till DynamoDB
6. Lambda returnerar data till API Gateway
7. API Gateway skickar svar tillbaka till webbläsaren
8. JavaScript uppdaterar sidan med data från DynamoDB

1.4 Skalbarhet

Automatisk skalning:

- S3 hanterar obegränsad samtidiga requests
- API Gateway skalar automatiskt
- Lambda exekverar parallellt (upp till account limits)

Kostnadseffektivitet:

- Betala endast för faktisk användning
 - Ingen kostnad för idle resources
 - Lambda free tier: 1M requests/månad
-

2. Säkerhet

2.1 S3 Bucket Security

Publikt läsbehörighet:

- Bucket policy tillåter `s3:GetObject` för alla (`Principal: "*"`)
- Nödvändigt för static website hosting
- Endast läsåtkomst - ingen skrivning eller borttagning tillåten

Encryption:

- Server-side encryption aktiverad (AES256)
- Data krypteras i vila automatiskt

Best practices implementerade:

```
PublicAccessBlockConfiguration:
  RestrictPublicBuckets: false    # Tillåter public web hosting
  BlockPublicPolicy: false       # Tillåter bucket policy
  IgnorePublicAcls: false
  BlockPublicAcls: false
```

2.2 API Gateway Security

Nuvarande konfiguration:

- Open API (ingen autentisering)
- Lämplig för publika read-only endpoints

Produktionsrekommendationer:

- Implementera API Keys för rate limiting
- Använd AWS WAF för DDoS-skydd
- Aktivera CloudWatch logging för audit trail
- Överväg Cognito för användarautentisering

2.3 Lambda Security

IAM Role: Lambda-funktionen har en execution role med minimal behörighet:

- CloudWatch Logs (för logging)
- DynamoDB Read Access (för att hämta svampdata)
- Inga extra permissions utöver nödvändiga

Best practices:

- Least privilege principle
- Automatiskt skapad role via CloudFormation
- Ingen hardkodad credentials i kod

Environment Variables:

- Inga känsliga data i koden
- Använd AWS Secrets Manager för credentials (vid behov)

2.4 CORS-konfiguration

Implementering:

```
'headers': {  
  'Access-Control-Allow-Origin': '*',  
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',  
  'Access-Control-Allow-Headers': 'Content-Type'  
}
```

Säkerhetsöverväganden:

- '*' tillåter alla origins - acceptabelt för publika APIs
- För produktionsmiljö: specificera exakta domains
- CORS är en browser security feature - ger inte server-side säkerhet

3. Infrastructure as Code

3.1 Metodik

Utvecklingsprocess:

1. Skapade resurser manuellt för att testa och validera
2. Genererade CloudFormation template via IaC Generator
3. Justerade och optimerade templaten
4. Raderade manuella resurser
5. Deployade via CloudFormation för reproducerbarhet

Fördelar med IaC:

- Versionskontroll av infrastruktur
- Reproducerbar deployment
- Enkel att återskapa miljön
- Dokumentation via kod

3.2 CloudFormation Templates

Projektet består av följande templates:

S3 Hosting:

- S3 Bucket med static website hosting
- Bucket policy för publikt läsårkomst
- Se: [Templates/s3-bucket.yaml](#)

Lambda och API Gateway:

- Lambda function med Python runtime
- IAM execution role med DynamoDB read permissions
- API Gateway REST API
- API Gateway resources, methods och deployment
- Lambda permissions för API Gateway invoke
- Se: [Templates/lambda-api.yaml](#)

DynamoDB:

- DynamoDB-tabellen ([Svampar](#)) skapades manuellt
- Partition key: [svampnamn](#) (String)
- Lambda-funktionen har IAM-permissions för att läsa från tabellen via [AmazonDynamoDBReadOnlyAccess](#) policy

Framtida förbättring: Implementera DynamoDB via CloudFormation för full IaC-deployment.

3.3 Deployment

Deployment via AWS CLI:

cd Templates

```
# Deploy S3 stack
aws cloudformation create-stack \
  --stack-name svampregistret-s3-stack \
  --template-body file://s3-bucket.yaml \
  --region eu-west-1

# Deploy Lambda + API Gateway stack
aws cloudformation create-stack \
  --stack-name svampregistret-api-stack \
  --template-body file://lambda-api.yaml \
  --capabilities CAPABILITY_IAM \
  --region eu-west-1
```

Post-deployment:

1. Hämta API Gateway URL från CloudFormation outputs
2. Uppdatera `index.html` med nya API URL
3. Ladda upp `index.html` till S3 bucket

4. Kostnadsanalys

4.1 Estimerad månadskostnad (låg trafik)

Tjänst	Användning	Kostnad/månad
S3 Storage	1 GB	\$0.023
S3 Requests	10,000 GET	\$0.004
API Gateway	10,000 requests	\$0.035
Lambda	10,000 invocations @ 128MB, 200ms	\$0.00 (free tier)
Total		~\$0.06

4.2 Skalningskostnader

Vid högre trafik (1M requests/månad):

- S3: ~\$0.40
- API Gateway: ~\$3.50
- Lambda: ~\$0.20
- **Total: ~\$4.10/månad**

Jämfört med traditionell hosting:

- EC2 t3.micro (always-on): ~\$8.50/månad
- Serverless är kostnadseffektivare vid låg/varierande trafik

5. Förbättringsområden

5.1 Säkerhet

- ☐ Implementera API authentication (API Keys eller Cognito)
- ☐ Begränsa CORS till specifika domains
- ☐ Aktivera AWS WAF för DDoS-skydd
- ☐ Implementera rate limiting

5.2 Prestanda

- ☐ Lägg till CloudFront CDN för global distribution
- ☐ Aktivera S3 Transfer Acceleration
- ☐ Optimera Lambda cold start (provisioned concurrency)
- ☐ Implementera caching i API Gateway

5.3 Monitoring

- ☐ Aktivera CloudWatch detailed monitoring
- ☐ Konfigurera CloudWatch alarms för fel och latency
- ☐ Implementera X-Ray för distributed tracing
- ☐ Sätt upp CloudWatch dashboards

5.4 CI/CD

- ☐ Automatisera deployment via GitHub Actions
- ☐ Implementera automated testing
- ☐ Blue/green deployment strategi
- ☐ Staging miljö för testning

6. Slutsats

Projektet demonstrerar en fungerande serverless arkitektur med:

- ☒ Static web hosting via S3
- ☒ Serverless backend via Lambda
- ☒ API management via API Gateway
- ☒ NoSQL-databas med DynamoDB
- ☒ Infrastructure as Code med CloudFormation
- ☒ Kostnadseffektiv och skalbar lösning

Lärdomar:

- Serverless passar utmärkt för event-driven applikationer
- IaC Generator förenklar CloudFormation-skapande
- CORS-konfiguration krävs för cross-origin requests
- Deployment automation är kritiskt för reproducerbarhet

7. Referenser och Dokumentation

Detaljerade instruktioner:

- [S3 Hosting Setup](#)
- [Lambda och API Gateway Tutorial](#)
- [DynamoDB Setup](#)
- [Lambda DynamoDB Integration](#)

CloudFormation Templates:

- [S3 Bucket Template](#)
- [Lambda + API Gateway Template](#)

GitHub Repository: <https://github.com/MariaSchillstrom/Skalbar-v-rdmilj--serverless-webapplikation>

Baserad på: Uppgiftsinstruktioner från kursen (med uppdateringar för AWS Console 2024/2025)

Alla filer, templates och detaljerade instruktioner finns tillgängliga i GitHub-repot.